UNIVERSITY OF THE ASSUMPTION

UniReserve · Campus Facility Reservation System

Technical System Architecture & Data Structures and Algorithms (DSA) Report · Desktop Edition (v1.0.0)

1. Executive Summary & Architecture Overview

UniReserve is a high-reliability, offline-first desktop application designed for the University of the Assumption (UA), San Fernando, Pampanga. It automates campus venue scheduling (Audio-Visual Room, Auditorium, Gymnasium, Covered Court) while enforcing formal conflict-free mathematical scheduling, strict role-based access control, and atomic offline database persistence.



2. Master DSA Integration Matrix

Every core feature in UniReserve is mapped directly to a classical Data Structure or Algorithm:

Component / Feature	Data Structure / Algorithm	Time Complexity	Space Complexity	Source Location
Conflict Detection	1D Interval Overlap Conjunction	O(K)	O(1)	`src/main/validation.js`
Database Indexing	Composite B+Tree Search	O(log N)	O(N)	`src/main/db.js`
Timetable View	2D Calendar Matrix Traversal	O(W × 7)	O(D)	`src/renderer/renderer.js`

Component / Feature	Data Structure / Algorithm	Time Complexity	Space Complexity	Source Location
Time Auto-Advance	Scalar Minute Math & Modulo	**O(1)**	O(1)	`src/renderer/renderer.js`
Status Lifecycle	Deterministic Finite State Automaton	**O(1)**	O(1)	`src/main/ipc.js`
Route Dispatch	Hash Map Table Lookup	**O(1)**	O(R)	`src/renderer/renderer.js`

ALGORITHMIC FOUNDATIONS & DATA STRUCTURES

UniReserve Computational Logic

3. The Interval Overlap & Conflict Detection Algorithm

The core mathematical guarantee of UniReserve is: **No two active reservations for the same facility on the same date may intersect in time.**

> **Mathematical Theorem:** Let existing reservation A be $[S_A, E_A)$ and candidate reservation B be $[S_B, E_B)$. The two intervals overlap if and only if:
>
> $$\text{Start}_A < \text{End}_B \quad \text{AND} \quad \text{End}_A > \text{Start}_B$$

```
Case 1: Disjoint Before (No Collision)
Existing A:   [08:00 --------- 10:00)
Candidate B:                           [13:00 ---- 15:00)
Condition:    (08:00 < 15:00 = TRUE) BUT (10:00 > 13:00 = FALSE) ==> VALID

Case 2: Back-to-Back Adjacent (No Collision)
Existing A:   [08:00 --------- 09:00)
Candidate B:                  [09:00 ---- 10:00)
Condition:    (09:00 < 09:00 is FALSE by strict inequality) ==> VALID

Case 3: Partial Collision (REJECTED)
Existing A:   [09:00 ---------------- 11:00)
Candidate B:            [10:30 ---------------- 12:30)
Condition:    (09:00 < 12:30 = TRUE) AND (11:00 > 10:30 = TRUE) ==> COLLISION DETECTED!

Case 4: Complete Containment / Nested (REJECTED)
Existing A:   [08:00 ------------------------------ 12:00)
Candidate B:            [09:30 -------- 10:30)
Condition:    (08:00 < 10:30 = TRUE) AND (12:00 > 09:30 = TRUE) ==> COLLISION DETECTED!
```

Actual Production Implementation (`src/main/validation.js`):

```
function isTimeOverlapping(startA, endA, startB, endB) {
  return startA < endB && endA > startB;
}

function checkBookingConflict(existingBookings, candidate) {
  for (const b of existingBookings) {
    if (b.status === 'Cancelled' || b.status === 'Rejected') continue;
    if (candidate.id && String(b.id) === String(candidate.id)) continue;
    if (isTimeOverlapping(candidate.startTime, candidate.endTime, b.start_time, b.end_time)) {
      return { hasConflict: true, conflictingBooking: b };
    }
  }
  return { hasConflict: false };
}
```

4. Database B+Tree Indexing Data Structure

Standard relational tables without indexes perform a sequential table scan: $mathcal{O}(N)$. As bookings grow to thousands of records, searching through every row causes perceptible UI stuttering.

UniReserve enforces a **Composite B+Tree Index** in SQLite (`idx_bookings_facility_date`). The index organizes the search keys into a self-balancing M-way search tree:

- **Tree Structure:** Root and internal nodes store pivot pairs `(facility_key, date)`; leaf nodes link directly to page data pointers.
- **Query Performance:** Lookups drop from $mathcal{O}(N)$ down to $mathcal{O}(log N + K)$, where K is the small set of bookings on that specific date.

5. 2D Calendar Matrix Traversal Algorithm

To display the monthly timetable, UniReserve builds a 2-dimensional matrix grid of dimensions $W imes 7$ (where W is the number of weeks in the month and 7 represents columns for Sunday through Saturday):

```
// Compute boundary dates to perfectly pad matrix rows (Sunday start)
const firstDay = new Date(year, month, 1);
const lastDay = new Date(year, month + 1, 0);

const startDate = new Date(firstDay);
startDate.setDate(startDate.getDate() - startDate.getDay()); // Pad Sunday offset

const endDate = new Date(lastDay);
endDate.setDate(endDate.getDate() + (6 - endDate.getDay()));  // Pad Saturday offset

// Traverse days into (row, col) matrix cells
while (current <= endDate) {
  const cellDateStr = formatDateLocal(current);
  if (cellDateStr === todayStr) td.classList.add('today');
  // Project matching bookings into the bucket
  const dayBookings = bookings.filter(b => b.date === cellDateStr);
  ...
}
```

TIME VECTOR MATH & PANEL DEFENSE CHEATSHEET

UniReserve Examination & Defense Guide

6. Time Vector Math & Auto-Advance Algorithm

To prevent invalid zero-minute or negative-duration reservations, the application integrates an automatic scalar minute projection algorithm:

1. **Deconstruction:** Parse 24-hour time `HH:MM` into total minutes from midnight: $T = (ext{Hours} imes 60) + ext{Minutes}$.
2. **Offset Addition:** Add the institutional minimum duration $Delta = 60$ minutes: $T_{ ext{target}} = T + 60$.
3. **Modular Reconstruction:** Compute new hours and minutes using integer division and modulo: $$ ext{Hour}_{ ext{new}} = leftlfloor rac{T_{ ext{target}}}{60} ight floor, quad ext{Minute}_{ ext{new}} = T_{ ext{target}} pmod{60}$$
4. **DOM Constraint Injection:** Set `reservationEndTime.min = autoEndTime` so users cannot submit an end time earlier than the minimum window.

7. Panelist Q&A / Defense Cheatsheet

Below are the most common questions asked by computer science panelists and faculty examiners, along with the precise technical answers:

Q1: Why did you use the interval overlap formula instead of checking every minute in a loop?

Answer: Checking minute-by-minute with a loop is $mathcal{O}(M)$ where M is total minutes, which is computationally wasteful and prone to off-by-one errors. The interval conjunction $(S_A < E_B land E_A > S_B)$ evaluates 1D range intersection in **$mathcal{O}(1)$ constant time** using boundary coordinates alone, making the verification instantaneous.

Q2: How does your database ensure fast retrieval when thousands of bookings exist?

Answer: We implemented a composite **B+Tree index** on `(facility_key, date)` in SQLite. Instead of a linear full table scan $mathcal{O}(N)$, the database performs a binary tree search in **$mathcal{O}(log N)$**, isolating only the active bookings for that facility on that single date.

Q3: How does the application handle timezones without shifting calendar dates?

Answer: Standard JavaScript `toISOString()` converts dates to UTC, which in the Philippines (GMT+8) shifts midnight back 8 hours to the previous calendar day. We engineered a custom `formatDateLocal()` algorithm that extracts local year, month, and day components directly, guaranteeing 100% calendar accuracy.

Q4: What data structure governs the lifecycle of a reservation request?

Answer: A **Finite State Automaton (State Machine)**. Requests begin in a `Pending` state, and transitions are strictly constrained to `Approved`, `Rejected`, or `Cancelled`. Only `Pending` and `Approved` states participate in conflict checking, allowing rejected or cancelled slots to be freed immediately.

Q5: Why is this application designed as an offline-first desktop app instead of a web app?

Answer: University venues often host operations during campus network outages or administrative isolation. UniReserve stores data in a local, embedded SQLite database with Write-Ahead Logging (WAL), offering zero-latency transactions and 100% availability regardless of campus internet connectivity.